



EXPRESS.JS CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. SETUP & APP

Installation

```
npm install express
import express from 'express'
const app = express()
app.use(express.json())
app.use(express.urlencoded({ extended: true }...
app.listen(3000, () => console.log('Running o...
```

04. REQUEST OBJECT

Workflow

```
req.params.id           // route :id
req.query.page          // ?page=2
req.body.email          // parsed JSON/form
req.headers.authorization
req.cookies.session     // (cookie-parser)
req.ip req.method req.path req.hostname
```

02. ROUTING

Architecture

```
app.get('/users', getUsers)
app.post('/users', createUser)
app.put('/users/:id', updateUser)
app.delete('/users/:id', deleteUser)
app.all('/admin/*', adminMiddleware, adminCon...
app.route('/items').get(list).post(create)
```

05. RESPONSE OBJECT

CRUD API

```
res.json({ users: [] })           // 200 JSON
res.status(201).json({ id: 1 })
res.status(404).json({ error: 'Not found' })
res.send('Hello')                 // text
res.sendFile(path)                // file
res.redirect('/login')
res.set('X-Header', 'value')
```

03. MIDDLEWARE

YAML / Config

```
app.use((req, res, next) => {
  console.log(req.method, req.url)
  next()           // call next middleware
})
app.use('/api', apiRouter)
app.use('/static', express.static('public'))
```

Order matters middleware runs top-to-bottom

06. ROUTER

Integration

```
// users.js
const router = express.Router()
router.get('/', listUsers)
router.post('/', createUser)
router.get('/:id', getUser)
export default router
// app.js
app.use('/api/users', usersRouter)
```



SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. ERROR HANDLING

Hardening

```
app.use((err, req, res, next) => {
  console.error(err.stack)
  res.status(err.status ?? 500).json({
    error: err.message
  })
})
```

4-arg middleware = error handler; must be last

next(err) to pass error to handler

10. BODY PARSING

Unit Tests

```
app.use(express.json({ limit: '10mb' }))
app.use(express.urlencoded({ extended: true }...
// Multipart / file upload:
import multer from 'multer'
const upload = multer({ dest: 'uploads/' })
app.post('/upload', upload.single('file'), ha...
```

08. ROUTE PARAMS & QUERY

Observability

```
app.get('/users/:id/posts/:postId', (req, res...)
const { id, postId } = req.params
const { page = 1, limit = 10 } = req.query
res.json({ userId: id, postId, page, limit })
})
app.param('id', (req, res, next, id) => {
  req.user = User.findById(id)
  next()
})
```

11. COMMON MIDDLEWARE

Commands

```
helmet()           // security headers
cors()             // CORS headers
morgan('dev')      // request logging
compression()      // gzip responses
cookie-parser      // parse cookies
express-rate-limit // rate limiting
express-validator  // input validation
```

09. STATIC FILES

Optimization

```
app.use(express.static('public'))
app.use('/assets', express.static('dist/asset...
// Serve with options:
app.use(express.static('public', {
  maxAge: '1d',
  etag: true
}))
```

Files in "public" served at root URL

12. COMMON GOTCHAS

Warning

Error handler must have 4 args (err, req, res, next)
Missing next() call hangs request indefinitely
Order of app.use() matters put catch-all last
express.json() body is undefined without middleware
async errors need try/catch + next(err) pattern